

RACTF 2020 Crypto - BOO7l3G CRYP70

deuterium — 2020-06-09

Canonical HTML: <https://deut-erium.github.io/WriteUps/2020/ractf/crypto/BOO7l3G'CRYP70/2020-06-09-RACTF-2020-BOO7l3G-CRYP70.html>

Abstract

CTF writeup: The random four-number blocks in RACTF 2020's BOO7l3G CRYP70 keep changing outfits but not their sums. Addition recovers the encoded characters.

B007l3G CRYP70

While doing a pentest of a company called MEGACORP's network, you find these numbers laying around on an FTP server:

```
41 36 37 27 35 38 55 30 40 47 35 34 43 35 29 32 38 37 33 45 39 30 36
27 32 35 36 52 72 54 39 42 30 30 58 27 37 44 72 47 28 46 45 41 48 39
27 27 53 64 32 58 43 23 37 44 32 37 28 50 37 19 51 53 30 41 18 45 79
46 40 42 32 32 46 28 37 30 43 31 26 56 37 41 61 68 44 34 26 24 48 38
50 37 27 31 30 38 34 58 54 39 30 33 38 18 33 52 34 36 31 33 28 36 34
45 55 60 37 48 57 55 35 60 22 36 38 34.
```

Through further analysis of the network, you also find a network service running. Can you piece this information together to find the flag?

Lets try to work out, what kind of ~encryption~ cipher it is.

Repeatedly encrypting the same message often gives out interesting pieces of information.

Lets encrypt a a few times

Welcome to MEGACORP's proprietary encryption service! Just type your message below and out will come the encrypted text!

Please enter the message you wish to encrypt: a
Your encrypted message is: 38 26 44 50

Please enter the message you wish to encrypt: a
Your encrypted message is: 44 48 39 27

Please enter the message you wish to encrypt: a
Your encrypted message is: 55 36 32 35

Please enter the message you wish to encrypt: a
Your encrypted message is: 46 29 28 55

Please enter the message you wish to encrypt: a
Your encrypted message is: 51 19 41 47

Please enter the message you wish to encrypt: a
Your encrypted message is: 41 59 25 33

Cool! there is some random element associated with encryption process, hence it is producing different ciphertext each time a is encrypted.

Lets check how encryption varies with length of provided plaintext,

```
Please enter the message you wish to encrypt: a
Your encrypted message is: 31 53 27 47

Please enter the message you wish to encrypt: aa
Your encrypted message is: 47 38 44 29 47 42 36 33

Please enter the message you wish to encrypt: aaa
Your encrypted message is: 29 54 37 38 31 53 33 41 40 53 50 15
```

Again, one can observe quickly that size of ciphertext is 4 times plaintext.

Now, with these kind of challenges, there is usually some kind of invariant associated with the ciphertext, which determines its association with corresponding plaintext.

My first guess was to try adding the four decimal values for encryption of a

```
38 + 26 + 44 + 50 = 158
44 + 48 + 39 + 27 = 158
55 + 36 + 32 + 35 = 158
46 + 29 + 28 + 55 = 158
51 + 19 + 41 + 47 = 158
41 + 59 + 25 + 33 = 158
```

NICE!! Lets verify if this invariant holds for other plaintext characters

```
Please enter the message you wish to encrypt: b
Your encrypted message is: 43 23 45 46

Please enter the message you wish to encrypt: b
Your encrypted message is: 20 58 29 50

43 + 23 + 45 + 46 = 157
20 + 58 + 29 + 50 = 157
```

Awesome, lets check how the invariant is carried across multiple blocks i.e. encryption of multiple plaintext characters

```
Please enter the message you wish to encrypt: aa
Your encrypted message is: 20 48 46 44 36 50 34 38

Please enter the message you wish to encrypt: aaa
Your encrypted message is: 26 25 56 51 33 43 59 23 44 32 45 37

20 + 48 + 46 + 44 == 158 and 36 + 50 + 34 + 38 == 158

26 + 25 + 56 + 51 == 158, 33 + 43 + 59 + 23 == 158 and 44 + 32 + 45 + 37 == 158
```

YAY! No more headaches, each character is encrypted independently and has nothing to do with any other letter. Lets write a script to decrypt.

We will first find the encryptions of all characters, calculate each invariants, then map each invariant to the ciphertext.

```
characters = "012346789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ{}  
[]!@#$%^&*()_+.,;: '\"  
encryptions = "72 45 46 44 53 44 51 58 50 45 65 45 81 32 49 42 45 70 41 47 32 48 64 57  
47 25 70 58 63 22 64 50 62 44 49 43 25 54 25 54 31 54 36 36 25 38 47 46 44 30 31 50 57  
32 33 32 44 34 37 38 45 21 46 40 48 29 38 36 49 37 39 25 23 43 49 34 33 44 35 36 28 34  
39 46 32 50 33 31 34 35 42 34 48 41 25 30 42 29 24 48 27 41 42 32 24 43 33 41 34 38 25  
43 38 47 19 35 45 45 25 23 41 21 33 42 43 35 24 34 35 35 30 35 39 23 38 34 23 29 37 44  
47 48 61 34 61 42 50 36 31 51 51 55 33 53 54 47 37 56 38 55 54 45 59 27 51 51 39 43 58  
34 29 62 31 52 48 51 53 29 35 64 53 57 18 52 63 30 42 44 71 32 33 42 29 53 46 49 32 44  
51 49 40 63 25 47 33 53 63 25 62 24 38 49 46 57 33 36 35 40 42 54 29 49 44 48 33 26 60  
50 63 31 32 42 48 26 48 45 47 38 28 53 25 43 36 61 19 45 40 28 26 35 33 36 46 49 20 49  
33 45 51 33 60 41 65 56 55 44 67 25 34 71 64 51 51 67 67 34 55 57 33 73 45 32 43 41 61  
51 68 37 61 67 46 39 45 41 56 73 52 65 48 49 40 38 38 44 30 66 70 46 52 54 55 50 68 47  
50 44 63 47 56 30 40 53 49 55 85 54 37 47 63 48 64 41 62 72 30 57".split()  
encryption_invariant = [ sum(map(int, encryptions[i:i+4])) for i in  
range(0,len(encryptions),4)]  
  
to_decrypt = "41 36 37 27 35 38 55 30 40 47 35 34 43 35 29 32 38 37 33 45 39 30 36 27  
32 35 36 52 72 54 39 42 30 30 58 27 37 44 72 47 28 46 45 41 48 39 27 27 53 64 32 58 43  
23 37 44 32 37 28 50 37 19 51 53 30 41 18 45 79 46 40 42 32 32 46 28 37 30 43 31 26 56  
37 41 61 68 44 34 26 24 48 38 50 37 27 31 30 38 34 58 54 39 30 33 38 18 33 52 34 36 31  
33 28 36 34 45 55 60 37 48 57 55 35 60 22 36 38 34".split()  
  
to_decrypt_invariant = [ sum(map(int, to_decrypt[i:i+4])) for i in  
range(0,len(to_decrypt),4)]  
flag = ""  
for value in to_decrypt_invariant:  
    flag += characters[encryption_invariant.index(value)]  
print(flag)
```

YESSS, we got our flag `ractf{d0n7_r0ll_y0ur_0wn_cryp70}`

A really fun challenge

| Interactive controls are omitted from this PDF. See the canonical HTML version.